

RV32I RISC-V SoC Design and Verification

Technical Documentation

Dasari Santhan Reddy

December 6, 2025

1 Project Summary

This project implements a custom **32-bit single-cycle RV32I RISC-V processor and System-on-Chip (SoC)** using Verilog HDL. The SoC integrates instruction memory, data memory, and a memory-mapped UART peripheral. Real RISC-V programs are compiled using the GNU RISC-V toolchain and executed on the custom core through RTL simulation. The design is also synthesized using Xilinx Vivado targeting an Artix-7 FPGA.

2 Key Features

- Single-cycle RV32I RISC-V processor core
- Byte-addressed instruction memory using `$readmemh`
- Data memory for load/store instructions
- Memory-mapped UART at address `0x00000080`
- Bare-metal RISC-V program execution
- RTL verification using Icarus Verilog
- FPGA synthesis using Xilinx Vivado

3 System Architecture

The system consists of the following major components:

- RISC-V Processor Core (`riscv_core`)
- Instruction Memory (IMEM)
- Data Memory (DMEM)
- UART Peripheral (`uart_sim`)
- Top-Level SoC Integration (`soc_top`)

The processor fetches instructions from instruction memory, executes them in a single cycle, and accesses memory or peripherals through memory-mapped I/O.

4 Processor Core Design

The processor core is implemented as a **single-cycle architecture** where each instruction completes in one clock cycle. The supported ISA is **RV32I**.

4.1 Major Submodules

- Program Counter (PC)
- Instruction Decoder and Immediate Generator
- Register File (32 × 32-bit)
- Arithmetic Logic Unit (ALU)
- Control Unit
- Branch and PC Update Logic

4.2 Register File

The register file contains 32 general-purpose 32-bit registers. Register `x0` is hardwired to zero according to the RISC-V specification. Two registers can be read and one register can be written in a single cycle.

4.3 Arithmetic Logic Unit

The ALU supports arithmetic and logical operations such as ADD, SUB, AND, OR, XOR, and comparison operations required for branch evaluation.

4.4 Control Unit

The control unit decodes the opcode and function fields of the instruction and generates the required control signals for:

- Register write-back
- Memory access
- ALU operation selection
- Branch decision

5 Memory System

5.1 Instruction Memory

Instruction memory is implemented as a **byte-addressed ROM**. RISC-V object code is converted into a Verilog-compatible HEX file using:

```
riscv-none-elf-objcopy -O verilog prog.elf program.hex
```

The HEX file is loaded using:

```
$readmemh("sw/program.hex", imem);
```

5.2 Data Memory

Data memory is implemented as a synchronous RAM module. It supports standard load and store operations from the processor core.

6 UART Peripheral

A simple UART transmitter is implemented as a **memory-mapped peripheral** at address:

```
0x00000080
```

When the processor writes to this address, the least significant byte of the data is transmitted and printed during simulation.

7 Software Toolchain

Bare-metal RISC-V programs are built using the following tools:

- `riscv-none-elf-gcc`
- Custom linker script
- `riscv-none-elf-objcopy`

7.1 Demo Program

A simple test program loads ASCII character ‘A’ into a register and writes it to the UART-mapped address, resulting in successful output during simulation.

8 Verification Methodology

RTL verification is performed using:

- Icarus Verilog
- GTKWave

Testbenches are written for:

- ALU
- Register File
- Decoder Control Unit
- RISC-V Core
- Complete SoC

Successful UART output confirms correct end-to-end execution.

9 Synthesis

The complete RTL design is synthesized using **Xilinx Vivado 2023.1** targeting:

Artix-7 (xc7a35t)

Synthesis verifies the hardware feasibility of the design.

10 Results

- Successful execution of real RISC-V programs on a custom processor.
- Correct UART output observed during simulation.
- Full integration of CPU, memories, and peripheral.
- RTL design successfully synthesized in Vivado.

11 Limitations

- Single-cycle architecture limits maximum operating frequency.
- No interrupt or exception handling.
- No cache memory.
- UART implemented only for simulation.

12 Future Enhancements

- Pipelined multi-stage RISC-V core
- Integration of timers and interrupt controller
- SPI, I2C, and GPIO peripherals
- Cache memory implementation
- FPGA board-level hardware testing

13 Conclusion

This project demonstrates the complete design, verification, and synthesis of a custom RV32I RISC-V SoC using Verilog. It validates the full software-to-hardware execution pipeline and provides strong practical exposure to computer architecture, RTL design, verification, and FPGA synthesis.

Repository

GitHub: <https://github.com/santhanreddy820>